

FormDrop v2

Redesign, monorepo, Elysia, SDKs

Everything in this document traces back to one conversation: a redesign in the direction of jam.dev and gumloop, a Turborepo monorepo, the public API on Elysia, docs on Fumadocs, PostHog analytics, email on SendByte, and first-party SDKs.

STATUS	DATE	OWNER
Decisions resolved	1 Sep 2026	devtofunmi
BRANCH	WORKSTREAMS	OPEN ITEMS
dev · 2d0e618	8	1 — D3

01

The brief

This PRD comes out of one conversation. Everything below traces back to something asked for in it — nothing has been invented, and nothing asked for has been dropped.

#	WHAT WAS ASKED	WHAT IT MEANS IN PRACTICE	STREAM
1	“I want to redesign formdrop.co”	New visual identity across the product	W4
2	Vibe references: jam.dev, gumloop.com — “lean more towards this style, especially the hero section”	Product-visual-forward hero, oversized type, chunky geometry, motion that demonstrates the product	W4
3	“Landing page and dashboard”	Both surfaces in scope; docs inherit the system	W4, W5
4	“Change the icon library to hugeicons”	Finish the half-done migration and make it enforceable	W4
5	“The codebase needs a refactor”	Route grouping, shared domain logic, no duplicated schema, tests	W3
6	“Set up as a monorepo using Turborepo”	pnpm workspaces plus Turborepo, apps and packages	W1
7	“The web app should stay as TanStack”	No framework change to the web app — an explicit non-goal	—
8	“The public API should be migrated to Elysia” (a backend framework)	public-api/ leaves Express for Elysia	W2
9	“Docs too. Use Fumadocs” (a documentation framework)	Hand-coded TSX docs routes become an MDX site	W5
10	“Set up analytics using PostHog” — “it’s more of a service than a framework”	An integration, not an architectural adoption: SDK wrappers and an event taxonomy, no product coupling	W6

#	WHAT WAS ASKED	WHAT IT MEANS IN PRACTICE	STREAM
11	"Emails will be moved from Resend to SendByte"	One provider, one set of templates	W7
12	"You'll work on the SDKs too"	First-party JS and React SDKs, published	W8

Two of these carry a distinction worth keeping visible, because it changes how much of the codebase each one touches.

Elysia and Fumadocs are frameworks. They replace something. Adopting them means real migration work with a parity bar and a cutover — W2 and W5 are the two riskiest streams in this document.

PostHog is a service. It is added, not migrated to. It sits behind a thin wrapper in `packages/analytics` so the product never depends on it directly and it can be swapped without touching a feature. Treating it as anything larger would be over-building.

02

Summary

FormDrop is the backend for headless forms: point a `<form>` (or a `fetch`) at a FormDrop endpoint, get submissions in a dashboard, notifications by email, Slack and Discord, and sync to Google Sheets and Airtable.

v2 moves on three fronts at once, which is why phasing matters more than usual here.

- **Design.** A new identity for the landing page and dashboard, in the direction of the two references. The hero is the anchor — it was called out specifically, and it is the piece everything else on the page has to live up to.
- **Platform.** The current layout — a TanStack Start app plus a standalone Express API carrying a *duplicated* database schema — becomes a Turborepo monorepo. The web app stays TanStack; the public API moves to Elysia; docs move to Fumadocs.
- **Plumbing.** PostHog for analytics, SendByte as the single email provider, and first-party SDKs so integrating FormDrop is one install instead of a hand-written `fetch`.

03

Non-goals

Stated so the redesign does not turn into a rewrite.

- **No web framework change.** TanStack Start stays, per direction. Elysia is for the public API only; the ~35 session-authenticated `src/routes/api/*` handlers stay in the web app.
- **No new product surface.** No form builder UI, no hosted form pages, no new integrations beyond what exists today.
- **No pricing or billing change.** Polar and the Free/Pro split stay as they are.
- **No auth change.** Better Auth stays.
- **No database migration.** Postgres and Drizzle stay. The schema moves into a package; it is not redesigned, except for the credential-hashing and delivery-log changes called out in W2 and W7.
- **No dark mode.** v2 ships a single light theme across landing, dashboard and docs (D5). Tokens are still structured so a dark palette can be added later without touching components.

- No SDK languages beyond JS and React (D4). No Python, no PHP/WordPress — not as a later phase either.

04 · W4

The redesign

This is the headline ask, so it comes first.

Reading the references

jam.dev and gumloop.com pull in the same direction, and that direction is what we adopt.

- **The product is on screen above the fold.** jam.dev leads with the artifact it produces; gumloop leads with its canvas. Neither leads with an illustration.
- **Oversized, tight-tracked type** as the primary visual device, with a short, concrete subhead. No adjective stacks.
- **Chunky geometry.** Large radii, generous padding, thick borders or soft shadows — surfaces that read as physical cards.
- **Restrained, confident colour.** One saturated accent doing a lot of work against a near-neutral ground. Gradients as atmosphere, not decoration.
- **Motion with a job.** Entrance choreography plus a looping animation that *demonstrates* something, rather than ambient float.
- **Developer-native proof.** Real code, copyable, in the first screen or the one right after.

Principles for FormDrop

1. **Show the drop.** The core loop — form submits, row lands in the dashboard, notification fires — is animatable, and it is the most characteristic thing this product does. The hero shows that loop end to end.
2. **One accent, deliberately.** Keep the violet `#6f63e4` as the brand anchor, formalized into a full 50–950 ramp instead of the `accent/10` opacity guesses used today.
3. **Tokens before components, components before pages.** Nothing in v2 ships a raw hex value or a bespoke button.
4. **One theme, done well.** v2 is light-only (D5). That is a licence to design *for* a light ground rather than compromising between two — but every colour still comes from a token, so dark stays a palette swap rather than a rewrite.
5. **Motion budget.** Entrances under 400 ms, one looping hero animation, `prefers-reduced-motion` respected everywhere.

The hero

The specific ask was to lean hardest here.

- **Headline:** two lines maximum, 64–88 px desktop, tight tracking. Replaces the current gradient-text treatment, which reads as a 2023 default.
- **Subhead:** one sentence, concrete, 20 words or fewer.
- **Primary CTA** filled in the accent, secondary to docs. Session-aware, as today.

- **Product visual:** an animated demo holding the visual mass below the fold line — a form on the left submitting into a dashboard row and a notification toast on the right, looping on a six-second cycle. A static, well-composed fallback under `prefers-reduced-motion`.
- **Background:** atmospheric. A soft accent wash and/or a subtle grid, masked at the edges. The current radial dot grid survives in spirit, re-tuned so it stops competing with the demo.
- The “Not backed by Y Combinator” badge stays. It is the only piece of voice on the page — re-treat it, do not remove it.
- The above-the-fold LCP element must be text, not the animation, so the page still scores well.

Landing page

```
Nav → Hero (animated loop) → Copyable snippet (HTML / fetch / SDK tabs)
  → How it works (3 steps) → Features → Integrations
  → Framework-agnostic proof → Pricing teaser → CTA → Footer
```

The existing `code-preview` component gains an **SDK tab** once `@formdrop/js` ships (W8) — the two streams meet here.

Dashboard

- **Shell.** The rounded sidebar is the right instinct; rebuild it on tokens with collapse-to-icons, a command palette for form switching, and a persistent form switcher in the header.
- **Forms list.** Cards become a dense table — last-7-day sparkline, submission count, integration status pills, and a real empty state containing the integration snippet.
- **Submissions.** The workhorse view: virtualized table, column visibility, saved views, keyboard row navigation, a detail drawer instead of a page navigation, bulk select to delete or export.
- **Analytics.** Keep Recharts, restyle on tokens, add per-form breakdown and previous-period comparison.
- **Notifications, integrations, settings.** Fold the three near-identical section components into one config-driven `IntegrationCard`.
- **Upgrade surface.** Keep the sidebar upsell card and move quota display into it.

Admin dashboard

In scope for the redesign (D7), not functional-only. The surface already exists — `admin.tsx`, `admin/index`, `admin/forms`, `admin/submissions`, `admin/settings`, `admin/users/index`, `admin/users/$userId` — and it should be brought onto the same primitives rather than left as the one screen that still looks like the old product.

- Same shell as the user dashboard, visually distinguished by a persistent “Admin” marker in the header, so there is never ambiguity about which surface an action lands on.
- **Overview** — platform totals: users, forms, submissions today / 7d / 30d, active subscriptions, notification failure rate. This is the screen that should answer “is the platform healthy” without opening the database.
- **Users** — searchable table, plan and status pills, submission volume, drill-through to a single user’s forms; keeps the existing `$userId` detail view.

- **Forms and submissions** — cross-tenant tables with the same virtualization and column controls as the user-facing versions, reusing the same components.
- **Settings** — keeps the existing maintenance actions, including clear-old-submissions, each behind an explicit confirm that names what will be deleted and how many rows.
- Admin routes carry a server-side role check, not just a hidden nav item.

Icons to Hugeicons

The migration is already 34 files in, with 9 files still importing `lucide-react`. Finish it and make it stick: remove the dependency, add a lint rule banning the import, and export one wrapped `<Icon>` from `packages/ui` so size and colour defaults live in a single place.

Files to convert: `auth-error`, `docs/code-block`, `docs/docs-sidebar`, `landing/features-grid`, `landing/integrations`, `sidebar`, `docs/index`, `docs/integrations`, `pricing`.

ACCEPTANCE

- Zero raw hex values outside `packages/ui` tokens.
- Lighthouse on `/`: performance 90+ mobile, accessibility 95+, CLS under 0.1, LCP under 2.0 s.
- Full keyboard traversal of the dashboard, visible focus rings, contrast 4.5:1 or better.
- `prefers-reduced-motion` disables the hero loop and every entrance animation.
- No `lucide-react` anywhere in the dependency tree.
- Admin routes use the shared primitives and enforce the role check server-side.

05 · W1

Monorepo with Turborepo

```
formdrop/
├─ apps/
│   ├─ web/           TanStack Start – landing + dashboard + admin (stays TanStack)
│   ├─ api/           Elysia – public API (api.formdrop.co)
│   └─ docs/          Fumadocs – MDX documentation
├─ packages/
│   ├─ db/            Drizzle schema, migrations, client ← single source of truth
│   ├─ core/          domain logic: submissions, notifications, integrations, quotas
│   ├─ email/         SendByte client + React Email templates
│   ├─ ui/            design tokens, primitives, Icon wrapper, Tailwind preset
│   ├─ analytics/     PostHog wrappers (browser + server)
│   └─ tsconfig/      shared tsconfig + lint config
├─ sdks/
│   ├─ js/            @formdrop/js
│   └─ react/         @formdrop/react
├─ turbo.json
└─ pnpm-workspace.yaml
```

- `pnpm-workspace.yaml` and a root `turbo.json` with `build`, `dev`, `lint`, `typecheck`, `test` pipelines and correct `dependsOn` / `outputs`. Move from npm to pnpm for the workspace protocol.

- Move the existing root app to `apps/web` unchanged — a mechanical move, no refactor in the same commit.
- Extract `packages/db` from `src/db`: schema, `auth-schema`, client, the `drizzle/` migrations, `drizzle.config.ts` and the `db:*` scripts. Delete `public-api/src/db`.
- Extract `packages/ui` — tokens and Tailwind preset first, primitives as W4 lands.
- Shared `packages/tsconfig`, ESLint flat config, Prettier.
- GitHub Actions running `turbo run lint typecheck build test` on PR, with remote caching.

Why this is the first phase. `src/db/schema.ts` and `public-api/src/db/schema.ts` are duplicated copies that are *already* on different Drizzle versions (0.39 vs 0.44). Every schema change today is a two-file change with silent drift, and every other workstream in this document touches the schema.

ACCEPTANCE

- `pnpm install && pnpm turbo build` builds every app from a clean clone.
- Exactly one `schema.ts` in the repo.
- A cached repeat build finishes in under 30 seconds.
- CI blocks merge on a typecheck failure.

Public API: Express to Elysia

New `apps/api` on Elysia, **running on Bun** (D1), with `@elysiajs/cors`, OpenAPI generation, and typed `t.Object` schemas on every route, so request validation stops being hand-rolled. **Parity first, then corrections.**

METHOD	PATH	AUTH	CHANGE FROM TODAY
GET	<code>/health</code>	—	replaces <code>GET /</code>
POST	<code>/f/:slug</code>	—	unchanged path — this is in the wild, it must never break
GET	<code>/v1/forms</code>	key	was <code>/forms</code>
POST	<code>/v1/forms</code>	key	NEW create a form via API
GET	<code>/v1/forms/:slug</code>	key	NEW
DELETE	<code>/v1/forms/:slug</code>	key	was <code>DELETE /forms/:formId</code> ; slug-addressed for consistency
GET	<code>/v1/forms/:slug/submissions</code>	key	was <code>GET /:slug/submissions</code> ; adds pagination
GET	<code>/v1/forms/:slug/submissions/:id</code>	key	NEW
DELETE	<code>/v1/forms/:slug/submissions</code>	key	bulk, body <code>{ ids: string[] }</code>
DELETE	<code>/v1/forms/:slug/submissions/:id</code>	key	—

CORRECTIONS CARRIED IN THE MIGRATION

- **Legacy aliases** for the five old authenticated paths stay **indefinitely**, undocumented but instrumented (D6). No sunset, no removal date — they cost one route definition each, and nothing breaks for anyone still calling them.
- **Pagination.** `GET /:slug/submissions` currently returns *every* submission for a form. Cursor-based (`?limit=50&cursor=`), `limit` capped at 200, response `{ data, nextCursor }`.
- **CORS.** Today `cors({ origin: "*" })` is applied to the whole API, including the API-key routes. Wide open belongs only on `POST /f/:slug` and `/health`.
- **Rate limiting.** Per-form on collect, per-key on `/v1/*`.
- **API key hashing.** Keys are stored in plaintext today, so a database read is a full credential leak. Store SHA-256, show the plaintext once at creation, keep a display prefix (`fd_live_...`).
- **Domain allowlist.** `isDomainAllowed` only runs when an `Origin` or `Referer` header is present, and falls back to a substring match — both are bypasses. Fix during the port and cover with tests.
- **The submission pipeline moves to** `packages/core` behind a durable outbox — a **Postgres job table** (D8), no new infrastructure. `POST /f/:slug` writes the submission, the usage counter and the outbox rows in one transaction, then returns. A worker drains the table with retry and exponential backoff, and rows that exhaust their retries stay visible as failures rather than disappearing. Today the fan-out is fire-and-forget: failures are logged and dropped, with no retries and no visibility for the user.
- Structured logging plus Sentry on unhandled errors, replacing `console.error`.

ACCEPTANCE

- Every legacy endpoint returns byte-compatible responses, proven by contract tests against the Express implementation.
- OpenAPI spec published and consumed by the SDKs.
- p95 on `POST /f/:slug` at or under 150 ms at 50 rps.
- A notification provider outage delays delivery but never loses a submission.

07 · W3

Codebase refactor

The ask was general; this is what it should mean concretely.

- **Route grouping.** `src/routes` currently mixes marketing, app, admin and docs in one tree. Reorganize into `(marketing)`, `(app)` and `(admin)` groups so layouts and auth guards stop being re-implemented per route.
- **Kill the hand-written client.** Replace the 400-line `src/lib/app-client.ts` axios wrapper, which re-declares by hand every type the database already knows, with thin typed clients whose types are derived from the handlers.
- **One place for domain logic.** Every `src/routes/api/*` handler calls `packages/core`; no direct Drizzle queries in route files.
- **De-duplicate UI.** Notification sections into `IntegrationCard`; `button`, `tooltip` and modals into `packages/ui`.
- **Type hygiene.** Fix the `(table: any)` casts in the schema during the `packages/db` extraction — they defeat Drizzle's index typing.

- **API-key UI** updated for hashed keys, with a show-once flow.
- **Tests.** Vitest is configured and there are currently **zero test files**. Units for `packages/core` (quota math, domain allowlist, notification routing) and a Playwright smoke path: signup, create form, submit, see submission.

ACCEPTANCE

- No component re-declares a type that already exists in `packages/db` or `packages/core`.
- `packages/core` at 70% line coverage or better; the domain-allowlist and quota functions at 100%.
- Playwright smoke suite green in CI.

08 · W5

Docs on Fumadocs

- `apps/docs` on Fumadocs, MDX content, built-in search, styled from `packages/ui` tokens. Fumadocs ships a dark theme by default — it gets disabled, so the docs match the rest of the product (D5).
- Port and expand the five existing hand-coded pages: Introduction, Getting Started, Forms, Integrations, API Reference.
- New content: SDK guides, framework recipes (Next.js, Astro, SvelteKit, plain HTML), notifications and webhooks, domain allowlisting, rate limits and quotas, error reference.
- **The API reference is generated from the Elysia OpenAPI spec**, not hand-written. Hand-authoring is exactly why today’s docs drift from the API.
- Served at `formdrop.co/docs` via a rewrite from `apps/web` (D2) — existing URLs keep working, and the docs stay on the primary domain for SEO. No subdomain.
- Delete `src/routes/docs*` and `src/components/docs/*`.

ACCEPTANCE

- Every current `/docs/*` URL resolves — 200 or 301.
- Search returns results for “API key”, “Slack”, “rate limit”.
- Regenerating the OpenAPI spec updates the reference with no hand edits.

09 · W6

Analytics with PostHog

Treated as a service integration, per the conversation — added behind a wrapper, not adopted architecturally.

- `packages/analytics` wrapping `posthog-js` (autocapture off — explicit events only) and `posthog-node`, with one `capture()` signature and `identify` on login wired to Better Auth. Nothing in the product imports PostHog directly.
- Reverse-proxy PostHog through `apps/web` so ad blockers do not erase the funnel.

- Event taxonomy, `object_action` snake case:
 - **Marketing:** `landing_viewed`, `hero_cta_clicked`, `pricing_viewed`, `docs_viewed`, `snippet_copied` (with `language`)
 - **Onboarding:** `signup_started`, `signup_completed`, `email_verified`, `form_created`, `first_submission_received`
 - **Core:** `submission_received`, `notification_sent`, `notification_failed` (with `channel`), `integration_connected` (with `provider`), `submissions_exported`, `api_key_created`
 - **Monetization:** `upgrade_modal_opened`, `checkout_started`, `subscription_activated`
- Dashboards: signup to first-submission funnel, activation by traffic source, notification failure rate by channel, free to pro conversion.
- Session replay on for `/app/*`, off for auth routes, with input masking.
- **Privacy.** Submission payload contents, recipient email addresses and IPs are never sent as event properties. `/privacy` is updated to disclose PostHog, with a consent path if EU traffic warrants one.

ACCEPTANCE

- The funnel renders with real data within 48 hours of launch.
- Server events fire from the Elysia API, not just the browser.
- A payload-content audit of captured events comes back clean.

10 · W7

Email: Resend to SendByte

The ask named Resend, but the repository is on **three** providers today, so the migration is wider than a swap.

WHERE	PROVIDER TODAY	WHAT IT SENDS
<code>src/lib/auth.ts</code>	Resend	OTP / email verification
<code>src/routes/api/forms/\$formId/recipients*</code>	Resend	Recipient verification
<code>public-api/src/lib/sendEmailNotification.ts</code>	ZeptoMail, with dead Plunk code behind a hardcoded <code>EMAIL_PROVIDER</code> constant	New-submission notification (inline HTML strings)

- `packages/email` exposing a single `sendEmail()` over SendByte, with `packages/email/templates` holding every template as a React Email component — the inline HTML strings go away.
- Templates: OTP / verification, recipient verification, new-submission notification, and the weekly/monthly report already promised on the pricing page.
- Delete the dead Plunk path and the `EMAIL_PROVIDER` constant; remove the `resend` and `zeptomail` dependencies.
- Log every send to a new `email_deliveries` table — provider message id, status, error — so failures are debuggable and can surface in the dashboard.
- Cutover: dual-send behind a flag in staging, verify DKIM/SPF/DMARC on the sending domain, flip production, keep the old credentials valid for seven days as rollback.

SendByte's API contract, SDK availability, sandbox mode, delivery webhooks and rate limits are **not specified in this document** — there is no verified reference for them here.

`packages/email` is therefore written behind our own `EmailProvider` interface, so the SendByte adapter is the only file that changes if the contract differs from expectations. W7 implementation should not start until the SendByte API docs are attached.

ACCEPTANCE

- Zero references to `resend` or `zeptomail` in the repo.
- OTP delivery success 99%+ over the first 1,000 sends, median delivery under 10 seconds.
- Every send has a row in `email_deliveries`.

11 · W8

SDKs

`@formdrop/js` ships first: zero dependencies, ESM and CJS, browser and Node, types generated from the OpenAPI spec.

```
import { FormDrop } from "@formdrop/js";

// public: submit (no key)
await FormDrop.submit("your-form-slug", { name, email });

// server: authenticated client
const fd = new FormDrop({ apiKey: process.env.FORMDROP_API_KEY });
const { data, nextCursor } = await fd.forms.submissions.list("contact", { limit: 50 });
```

It owns endpoint construction, `FormData` and JSON bodies, error normalization into typed error classes, retries with backoff on 5xx and 429, and an `AbortSignal` pass-through.

`@formdrop/react` is a hook over it:

```
const { submit, isSubmitting, isSuccess, error, reset } = useFormDrop("your-form-slug");
```

Plus an optional `<FormDropForm>` wrapper handling `onSubmit`, serialization and a honeypot spam field.

Scope is these two packages only (D4). No Python, no PHP/WordPress — the landing page already argues FormDrop is framework-agnostic, and for anything outside the JS ecosystem a plain `POST` to `/f/:slug` is the documented answer, covered by the framework recipes in W5.

ACCEPTANCE

- Versioned independently, published from CI on tag, with `provenance: true`.
- Contract tests run against a live staging API in CI, so an API change breaks the SDK build rather than users.
- `npm i @formdrop/js` to first successful submission in under two minutes, verified by a fresh-machine walkthrough of the quickstart.
- Bundle size: `@formdrop/js` at or under 5 KB gzipped, `@formdrop/react` at or under 2 KB on top.

12

Phasing

Ordering rationale: **platform before paint wherever the paint depends on it**. Tokens can start immediately, but dashboard screens should not be rebuilt twice — and the stability of the collect endpoint outranks everything else.

PHASE	CONTENTS	EXIT CRITERIA
P0 · Foundation	W1 monorepo, <code>packages/db</code> , CI; <code>packages/ui</code> tokens; PostHog baseline instrumentation on the <i>current</i> landing page	Clean-clone build green; baseline conversion numbers recorded
P1 · Hero + identity	W4 tokens, primitives, hero , landing page; Hugeicons completion	New landing live; Lighthouse targets met
P2 · API	W2 Elysia parity, <code>packages/core</code> extraction, outbox worker, contract tests, legacy aliases instrumented	Elysia serving production traffic; Express deleted
P3 · Dashboard	W3 refactor plus W4 dashboard shell and screens; admin dashboard	All <code>/app/*</code> and <code>/admin/*</code> on primitives; smoke suite green
P4 · Docs + SDKs	W5 Fumadocs at <code>/docs</code> ; W8 <code>@formdrop/js</code> and <code>@formdrop/react</code> ; OpenAPI-generated reference	Docs live with no dead URLs; both SDKs published
P5 · Email + analytics depth	W7 SendByte cutover; W6 dashboards, replay, funnels	Single provider; funnels reporting

W6 deliberately spans P0 and P5: instrumenting only *after* the redesign would leave us unable to say whether the redesign worked.

13

Risks

RISK	IMPACT	MITIGATION
<code>POST /f/:slug</code> breaks during the Elysia migration	Customer forms silently stop collecting — the worst failure mode this product has	Path unchanged; contract tests against Express; canary by percentage; a synthetic monitor posting to a real form every 60 s, with alerting
Redesign, refactor and migration land together and nothing is attributable	Cannot separate a regression from a redesign effect	Phased launch; PostHog baseline captured before P1

RISK	IMPACT	MITIGATION
SendByte contract differs from assumptions	W7 rework	EmailProvider interface; W7 gated on the API docs (D3, still outstanding)
Bun in production is new operational ground	Deploy and runtime surprises	Decided (D1), so this is a preparation task rather than an open question: load-test on Bun in staging before P2 exits, pin the Bun version in CI and the Dockerfile, and confirm the host supports it before the port starts
Fumadocs (Next.js) inside a Vite/TanStack monorepo	Build friction, two React versions	Isolate as its own workspace app, no shared React runtime; verify packages/ui tokens are consumable as plain CSS
Scope creep from “redesign” into new features	Timeline	Non-goals enforced in review
Zero test coverage at the start of a large refactor	Regressions ship unnoticed	Tests are acceptance criteria on W2 and W3, not a follow-up
Existing plaintext API keys cannot be recovered once hashing lands	Users’ keys break	Hash-on-next-use dual-read window, then forced rotation with in-app and email notice

14

Success metrics

PRODUCT

+30% Landing to signup conversion, against the P0 baseline	50% Signups with a first submission within 24 hours	<10 min Median time from signup to first submission	25% Docs to SDK quickstart to first submission, funnel completion
--	---	--	---

PLATFORM

- p95 on POST /f/:slug at or under 150 ms; availability 99.9%+.
- Notification delivery success 99.5%+, with failures visible in the dashboard.
- Cached CI build at or under 30 seconds, cold at or under 4 minutes.
- packages/core coverage 70%+.
- Zero duplicated schema definitions; zero plaintext credentials at rest.

15

Decisions

Resolved 1 September 2026. These are settled, not open for re-litigation in review; each one is already reflected in the workstream it affects.

#	DECISION	CONSEQUENCE
D1	API runtime: Bun	apps/api targets Bun. @elysiajs/node is off the table, so the host must support Bun — confirm before P2 starts

#	DECISION	CONSEQUENCE
D2	Docs at <code>formdrop.co/docs</code>	Rewrite from <code>apps/web</code> ; no subdomain, no redirects needed, existing URLs survive
D4	SDKs: JS and React only	<code>@formdrop/js</code> and <code>@formdrop/react</code> . Python and PHP are out of scope entirely, not deferred
D5	No dark mode	Single light theme everywhere; Fumadocs' default dark theme gets disabled; one palette to design and test
D6	Legacy aliases kept indefinitely	No sunset, no deprecation notice; they stay instrumented so usage is visible if we ever revisit
D7	Admin dashboard in scope	<code>/admin/*</code> is redesigned on the shared primitives (\$04), landing in P3 alongside the user dashboard
D8	Postgres outbox table	No queue infrastructure; a table drained by a worker, written in the same transaction as the submission

Still outstanding

D3 • BLOCKS P5

SendByte API docs. Needed before W7 can start: base URL, auth scheme, send payload shape, template support, delivery webhooks, sandbox/test mode, rate limits. The user will provide these.

Until then `packages/email` is built against our own `EmailProvider` interface with the existing providers behind it, so the SendByte adapter is the only file the real contract touches.

One thing to confirm

D6 was answered “no need”. This PRD reads that as *no need for a sunset* — the aliases stay forever. If the intent was the opposite (no need for the aliases at all, drop them at the cutover), say so: that turns the Elysia migration into a breaking change for any existing API-key consumer, and the five alias routes come out of W2.

Current state

Read from the repository at `2d0e618`. This is the evidence behind the workstreams above.

Layout. A single npm project at the root (TanStack Start, Vite 7, React 19, Tailwind 4) plus a nested, separately-installed `public-api/` on Express 5. No workspaces, no shared packages, no CI config, roughly 16,500 lines of TS/TSX, zero test files.

Existing public API surface

METHOD	PATH	AUTH	NOTES
GET	<code>/</code>	—	version string
POST	<code>/f/:slug</code>	—	collect a submission; fans out notifications and Sheets sync
GET	<code>/forms</code>	API key	list forms
DELETE	<code>/forms/:formId</code>	API key	soft delete
GET	<code>/:slug/submissions</code>	API key	unbounded — returns every submission
DELETE	<code>/:formId/submissions</code>	API key	bulk soft delete
DELETE	<code>/:formId/submissions/:submissionId</code>	API key	soft delete

Design. Tailwind 4 with exactly one custom token (`--color-accent: #6f63e4`); everything else is inline utility classes. Inter via a Google Fonts `@import`. Light mode only. Icons: 34 files on Hugeicons, 9 still on `lucide-react`.

Problems the workstreams address

- 01 Duplicated schema on two different Drizzle versions. W1
- 02 No pagination on the submissions endpoint. W2
- 03 Inconsistent addressing — some routes key on `slug`, others on `formId`, and submissions routes are mounted at the API root where they can collide with future top-level resources. W2
- 04 API keys stored in plaintext, unique and indexed. W2
- 05 OAuth tokens stored in plaintext — Google Sheets and Airtable access and refresh tokens sit on `forms`. W2 FOLLOW-UP
- 06 `cors({ origin: "*" })` across the whole API, API-key routes included. W2
- 07 Bypassable domain allowlist — header-dependent, with a substring fallback. W2

08 **Fire-and-forget notification fan-out** — no retries, no dead-letter, no user-facing delivery visibility.

W2

09 **Usage accounting inside the request, after the fan-out** — adds latency, fails independently of the submission. W2

10 **(table: any)** casts throughout the schema, defeating Drizzle's index typing. W3

11 **Three email providers** across two projects, one of them dead code. W7

12 **No product analytics** — the **events** table logs domain events only. W6